



Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación
IIC2523 – Sistemas Distribuidos

Tarea 3

Simulación de transacciones Distribuidas

Fecha de inicio: **Martes 14 de octubre** a las **20:00:00** hrs

Plazo máximo para **formalizar parejas**: **Miércoles 22 de octubre** a las **20:00:00** hrs.

Formulario de inscripción: <https://forms.office.com/r/1F05AQYCFG> Es obligatorio responder este formulario si trabajarás con otro estudiante. En otro caso, se asumirá que trabajarás de forma individual y no se aceptará reclamos al respecto.

Fecha de entrega: **Domingo 26 de octubre** a las **20:00:00** hrs.

Fecha máxima de entrega (con atraso): **Martes 28 de octubre** a las **20:00:00** hrs.

Evaluación en el contexto del curso

Esta es una evaluación **de corrección automática** y con la opción de realizarla de forma **individual** o **en parejas** si se inscriben en el formulario. Además, pretende rescatar evidencias del desarrollo de ciertos resultados de aprendizaje. Por eso, tras su realización y entrega, recibirás retroalimentación respecto a los diferentes *tests* que se esperaba que la tarea lograra exitosamente. En particular, los objetivos específicos de esta evaluación son:

- Representar transacciones que ocurren de forma concurrente
- Aplicar correctamente un protocolo de *commit* basado en quórum y en 2 fases.
- Evaluar el efecto de operaciones incompletas (por ejemplo, *commits* sin quórum, can-commit pendientes) sobre el estado del sistema.

Contexto

Al realizar un balance del semestre, resulta imposible pasar por alto ciertas experiencias poco convencionales. Entre ellas, la construcción forzada de un servidor distribuido con el insólito propósito de calificar “monos chinos”, seguida de una investigación más propia de una serie policial que de un curso universitario, con múltiples escenas del crimen y testimonios contradictorios. Sin duda, ha sido un semestre intenso... y lo sorprendente es que aún no concluye.

En ese contexto aparece un nuevo giro narrativo: Guaripolo —sí, el personaje favorito de los niños de 31 Minutos— se presenta junto a Juan Carlos Bodoque con una propuesta inesperada, digna de un caso de estudio en sistemas distribuidos: la creación del primer banco en Titirilquén con alcance internacional.

Aunque la idea inicial parecía carecer de toda seriedad, el planteamiento pronto reveló una intención técnica clara. Bodoque explicó que este nuevo banco debía estar soportado por una arquitectura distribuida, con servidores desplegados en distintas regiones del país para minimizar la latencia. A ello añadió una preocupación fundamental: la necesidad de preservar la consistencia de los datos, incluso en un sistema que puede ser propenso a fallas y concurrencia.

De esta forma, se establecieron dos lineamientos innegociables:

1. Todas las modificaciones de la base de datos deben realizarse en forma transaccional.
2. Cada operación debe emplear el protocolo de dos fases (2PC) y ejecutarse solo tras alcanzar un quórum.

No obstante, faltaba definir el mecanismo de control de concurrencia a aplicar. La opción pesimista fue descartada bajo el poderoso argumento técnico de Guaripolo: “suena muy deprimente”. Por lo tanto, solo persiste el dilema metodológico: ¿cuál tipo de validación, en el control de concurrencia optimista, resulta más conveniente: *forward* o *backward*? Dado lo anterior, el problema pendiente exige comparar, experimentalmente, ambos tipos de validación bajo distintos escenarios de transacciones distribuidas.

Consciente de la magnitud de la tarea, Bodoque gestionó un acuerdo con el profesor que consiste en 2 puntos. Primero, tu sub-contratación como investigador adjunto, remunerada exclusivamente con una nota en el curso; y segundo, otorgar clases privadas al profesor respecto a cómo apostar y ganar en carreras de caballos. El docente, sin ideas frescas para nuevas tareas y con evidente interés en la segunda parte del trato, aceptó sin reparos.

Así, tu misión académica en esta última tarea de programación del curso consiste en desarrollar un simulador de transacciones distribuidas que incorpore el protocolo de dos fases y quórum en cada modificación, permitiendo contrastar experimentalmente los enfoques optimistas *forward* y *backward*. El destino del primer banco internacional de Titirilquén —y de la última nota de tarea— dependerá de los resultados de tu simulación.

Introducción

En esta evaluación deberás desarrollar un simulador de transacciones distribuidas que permita experimentar con diferentes enfoques de control de concurrencia optimista: *forward* y *backward*. El sistema deberá garantizar en todo momento la ejecución de operaciones bajo el protocolo de dos fases (2PC), así como el cumplimiento de un quórum antes de aplicar cualquier transacción a la base de datos.

El objetivo principal es analizar, mediante simulación, cómo se comportan ambos mecanismos de control de concurrencia en escenarios con múltiples transacciones simultáneas. Para ello, se espera que tu solución sea capaz de simular las distintas situaciones de conflicto y registrar los resultados obtenidos aplicando las reglas de 2PC con quórum, y validación *backward* y *forward*.

Toda la tarea debe ser desarrollada con Python 3.X con $X \leq 12$.

1. Transacciones Distribuidas

En esta tarea vamos a simular la ejecución de múltiples transacciones de forma concurrente, las cuales intentarán leer y modificar una base de datos. Para lograr esta simulación, cada archivo de *test* contendrá la siguiente información:

- Un diccionario con la base de datos que tiene únicamente las variables consolidadas.
- Una lista con el nombre de los servidores disponibles para utilizar el protocolo 2PC.
- El tipo de validación a utilizar en la simulación (*backward* o *forward*).
- Una lista de eventos que ocurren durante la simulación, los cuales deberán ser ejecutados de forma secuencial y en el mismo orden que están en dicha lista. El formato de los eventos será explicado en la sección [Formato de eventos de la simulación](#)

Respecto a los eventos, estos corresponden a los comandos aprendidos en clases: *BEGIN*, *READ*, *WRITE*, *COMMIT*. No obstante, se agregan 3 eventos más: *CAN_COMMIT*, *READ_COMMIT* y *READ_POSSIBLE_VALUES*. Cada uno de estos eventos será explicado en la sección [Comandos y Consultas](#).

Respecto a las transacciones, cada una de estas presentará un identificador único que empezará siempre con una *T* mayúscula, el cual será utilizado en los archivos de simulación para poder identificar cada comando con la transacción que la está aplicando. Además, las transacciones deben alinearse a las propiedades ACID. Por este motivo, al momento de explicar los comandos, se detallará una serie de condiciones que se deben seguir para la correcta ejecución de cada comando.

Finalmente, cada transacción posee un estado, el cual puede ir cambiando según los comandos que esta vaya ejecutando y la respuesta que se obtenga de dicha ejecución. Los estados posible son:

- **ABIERTA**: corresponde al estado por defecto que tiene la transacción cuando se hace por primera vez el comando *BEGIN*.
- **EN_PREPARACION**: es el estado que tiene la transacción cuando realiza, por primera vez, el comando *CAN_COMMIT* del protocolo 2PC y dicho comando se ejecutó con éxito.
- **CONFIRMADA**: es el estado que tiene la transacción cuando realiza el comando *COMMIT* del protocolo 2PC y el comando se ejecuta con éxito.
- **ABORTADA**: es el estado que tiene la transacción cuando realiza el comando *ABORT*.
- **INVALIDA**: es el estado que tiene la transacción cuando se detecta una inflación que hace imposible su continuación en el sistema.

2. Comandos y Consultas

Durante la simulación, existirán una serie de eventos que pueden ser catalogados como comandos o consultas. Los comandos son acciones realizadas por las transacciones, mientras que las consultas son acciones realizadas por la simulación para saber el valor de alguna variable en la base de datos.

2.1. Comandos

Los comandos corresponden a las diferentes acciones que puede realizar una transacción. Todo comando va a ir acompañado del identificador de la transacción el cuál lo está aplicando. No obstante, algunos comandos puede requerir algún argumento adicional. Los comandos que debe soportar la simulación son:

- **BEGIN**

Esta acción no recibe ningún argumento. Su función es inicializar dicha transacción para empezar a procesar cualquier otro comando. Este comando solo tiene efecto la primera vez que aparece por transacción y si llega a aparecer nuevamente en una transacción que ya hizo **BEGIN**, se debe ignorar dicho comando y no tener repercusión en la simulación.

Cualquier acción realizada por una transacción antes de que fue inicializada es ignorado por el sistema.

- **WRITE(NOMBRE_VARIABLE: str, VALOR: str)**

Esta acción recibe 2 argumentos, el nombre de una variable y el valor a guardar; y su función es escribir por primera vez o sobre-escribir, la variable **NOMBRE_VARIABLE** con el valor indicado en **VALOR** en la base de datos. Para esta acción, debemos recordar que la transacción debe alinearse a las propiedades ACID, en particular a la propiedad A e I (*atomicity e insolation*). Por lo tanto, la modificación realizada a la base de datos solo existe dentro de la transacción y esta se verá reflejada en la base de datos únicamente si la transacción es confirmada con éxito.

Adicionalmente, si **VALOR** es exactamente **DELETE**, entonces el **write** está eliminando la variable de la base de datos. Esta acción se ve reflejada internamente en la transacción de forma inmediata, es decir, para la transacción ya no existe esa variable dentro de la base de datos de la transacción. Luego, una vez confirmada la transacción exitosamente, si el **WRITE** con valor **DELETE** fue la última modificación realizada a la variable **NOMBRE_VARIABLE**, entonces dicha variable es eliminada de la base de datos. Si se intenta eliminar una variable que no existe en la base de datos, entonces este comando se ignora y no genera ningún error.

Finalmente, si una transacción ejecuta el comando **WRITE** cuando su estado es **EN_PREPARACION**, el comando no debe ser procesado y la transacción queda marcada como **INVALIDA**.

- **READ(NOMBRE_VARIABLE: str)**

Esta acción recibe 1 argumento, el nombre de una variable. Su función es acceder a la variable **NOMBRE_VARIABLE**. Para esta acción, debemos recordar que la transacción debe alinearse a las propiedades ACID, en particular a la propiedad I (*insolation*). Por lo tanto, la lectura de la variable, en primera instancia, será realizada sobre la base de datos que existe dentro de la transacción. Luego, si esta variable no existe en dicha base de datos, se intentará obtener su valor de la base de datos que tiene solo las variables consolidadas.

Si alguna variable no existe en los dos casos mencionados anteriormente, entonces el comando no es procesado y esta transacción es marcada como **INVALIDA**, dado que intentó obtener el valor de una variable que no existe.

Por otro lado, al igual que el comando **WRITE**, si una transacción ejecuta el comando **READ** cuando su estado es **EN_PREPARACION**, el comando no debe ser procesado y la transacción queda marcada como **INVALIDA**.

■ **CAN_COMMIT(NOMBRE_SERVIDOR: str)**

Esta acción recibe 1 argumento, el nombre del servidor donde se ejecutará el comando. Su función corresponde a simular la primera fase del protocolo 2PC, es decir, solicitar al servidor **NOMBRE_SERVIDOR** que valide si la transacción puede hacer *commit* en el futuro y en caso que sea posible, reservar las variables correspondientes hasta que se confirme el *commit*. Para esto, **se realizan 2 validaciones**:

- **Control de concurrencia**: Se verifica que cumpla con el método de validación correspondiente: *backward* o *forward* para evitar conflictos del **tipo read-write**. Para esto, el servidor tendrá acceso a todas las transacciones válidas que existen a la fecha (las confirmadas y las que todavía no se confirman) y utilizará el método de validación correspondiente a la simulación para dar una respuesta.
- **2PC**: En caso que el servidor ya haya aceptado previamente otra transacción con **CAN_COMMIT**, se debe validar que la transacción no genere conflictos del **tipo read-write** con las variables correspondientes que fueron reservadas en la primera fase del protocolo 2PC.

En caso que ambas validaciones sean exitosas, el servidor acepta la transacción. Por lo tanto, el servidor protegerá las variables que se utilizan en la transacción para garantizar que no existan conflictos del tipo *read-write*.

Por otro lado, en caso de que la validación no sea exitosa, pueden ocurrir 2 situaciones:

1. Si el motivo de rechazo fue por el protocolo 2PC o por utilizar la validación *forward*, el servidor solamente responde que la transacción no será aceptada, pero no hace nada más con la transacción, es decir, no la obliga a abortar y tampoco cambia el estado actual de la transacción.
2. En cambio, si el motivo de rechazo fue por utilizar la validación *backward*, el servidor obliga a que la transacción sea abortada inmediatamente para asegurar la consistencia en todo el sistema. Por lo tanto, se ejecuta, de forma interna, el comando **ABORT** con la transacción que fue rechazada.

Finalmente, desde el momento que se hace el primer **CAN_COMMIT** y este es aceptado, la transacción pasa a estar en estado **EN_PREPARACION** y ya no puede hacer ningún **READ** o **WRITE** adicional o será declarada como inválida. Además, en caso que un servidor ya haya aceptado el **CAN_COMMIT** y reciba nuevamente la solicitud de la misma transacción, ignora la petición.

■ **ABORT**

Esta acción no recibe ningún argumento. Su función es abortar la transacción de forma definitiva, es decir, se cancelan todos los **WRITE** y **READ** que hayan realizado haciendo como si nunca hubieran ocurrido en el sistema. Si algún servidor tiene la transacción aprobada por el protocolo 2PC, también es abortada y las variables bloqueadas en dicho servidor son liberadas.

Finalmente, toda acción realizada posterior a que la transacción fue abortada debe ser ignorada por el sistema.

■ COMMIT

Esta acción no recibe argumentos. Se encarga de consolidar la transacción de forma definitiva en todos los servidores, incluso en los que no participaron del `CAN_COMMIT`. Esta acción solo se ejecuta exitosamente si se cumplen las 3 condiciones al mismo tiempo:

1. Se logró quórum en el `CAN_COMMIT`, es decir, la transacción fue aceptada, mediante el comando `CAN_COMMIT`, por la mayoría de los servidores ($\lfloor N/2 \rfloor + 1$ servidores).
2. La transacción sigue siendo válida, es decir, no hizo ningún `READ` y `WRITE` posterior a realizar exitosamente el primer `CAN_COMMIT`.
3. Se realiza una última validación del tipo *backward* y esta sale exitosa.

En caso de no cumplir la condición 3, al igual que en `CAN_COMMIT`, el servidor obliga a que la transacción sea abortada de todo el sistema.

En caso de no cumplir las condiciones 1 o 2 (pero sí la 3), no se consolida la transacción. No obstante, la transacción no sufre ninguna repercusión por ejecutar este comando, permitiéndole volver a intentar el comando nuevamente más adelante, es decir, no es obligada a abortar o ser declarada inválida, simplemente se ignora la petición.

En caso de cumplir las 3 condiciones, la transacción es distribuida a todos los servidores y todos los `WRITE` de dicha transacción son consolidados, es decir, los cambios que hizo la transacción son aplicados a la base de datos distribuida para que toda nueva transacción vea esos datos. Además, si existe uno o más servidores que tienen alguna transacción aceptada por `CAN_COMMIT`, pero dicha transacción lee alguna variable que la transacción consolidada hizo `WRITE`, entonces esa transacción aceptada por `CAN_COMMIT` debe ser abortada automáticamente de todo el sistema.

Finalmente, toda acción realizada posterior a que la transacción fue consolidada debe ser ignorada por el sistema.

2.2. Consultas

Además de los comandos ejecutados por las transacciones, la simulación debe soportar consultas que permitan observar el estado de las variables de la base de datos en diferentes momentos de la ejecución.

Estas consultas no modifican el estado del sistema, sólo entregan información, además, el resultado de ellas son registradas en el archivo de salida bajo la sección LOGS, en el mismo orden en que aparecen en el archivo de entrada. En la sección de [Resultados de la simulación](#) se detalla cómo guardar los datos de estas 2 consultas.

- `READ_POSSIBLE_VALUES(NOMBRE_VARIABLE: str)`

Devuelve el conjunto de todos los valores posibles que la variable `NOMBRE_VARIABLE` puede tomar en ese instante de la simulación. Este conjunto se forma a partir del valor consolidado en la base de datos, junto con los valores escritos en transacciones se encuentran en estado ABIERTA o EN_PREPARACION. Si no existen valores posibles, la consulta debe retornar un conjunto vacío.

En caso que una transacción eliminó la variable (`WRITE` con valor `DELETE`) justo antes que se hace la consulta, entonces dicha transacción no aporta con ningún posible valor al conjunto de todos los posibles valores.

Al momento de guardar este conjunto, este se debe transformar en un lista en formato JSON (`json.dumps(list(conjunto))`) cuyo orden de sus elementos queda a criterio del programador.

- `READ_COMMIT(NOMBRE_VARIABLE: str)`

Devuelve el valor consolidado de la variable `NOMBRE_VARIABLE` en la base de datos al momento de ejecutar la consulta. Si la variable no existe, la consulta debe retornar el texto `NULL`.

Al momento de guardar este valor, se debe escribir directamente el valor o la palabra `NULL` sin ninguna comillas.

2.3. Supuestos y reglas generales

Sin importar el mecanismo de control de concurrencia que se ejecute, hay ciertos supuestos y reglas que debes tener en cuenta en la implementación que desarrolles:

- El *script* desarrollado debe ser capaz de procesar archivos que incluyan comentarios (`.jsonc`). Estos comentarios deberán ser ignorados al momento de procesar la información. Un comentario inicia siempre con los 2 caracteres de *forward slash* (`//`), los cuales pueden estar en cualquier parte del archivo, el comentario debe ser ignorado sin afectar la información del archivo. Se recomienda aplicar Regex para eliminar los comentarios previo a abrir el archivo con la librería JSON.
- Puedes asumir que ningún otro *string* contendrá 2 *forward slash* (`//`) seguidos, como tampoco contendrá el carácter de punto y coma (`;`). Estos estarán reservados para otros usos.
- Cada archivo `.jsonc` debe procesarse de forma independiente. El estado de la base de datos no debe compartirse entre simulaciones.
- **NO puedes asumir que los servidores utilizados en los comandos de las transacciones correspondan a uno de los definidos inicialmente.** Por ejemplo, un servidor se puede llamar `ServerTulioTriviño` y en un comando se usa el servidor `servertuliotriviño`, es decir, son *strings* distintos. En caso de ocurrir que el servidor no exista, ese comando debe ser ignorado y no tener ningún efecto en la simulación.
- Puedes asumir que los archivos `.jsonc` siempre cumplen con el formato descrito y que **fueron creados con un *encoding* de UTF-8.**
- La ejecución de cada evento debe **seguir estrictamente las reglas y restricciones específicas de cada mecanismo y algoritmo** visto en el curso.

2.4. Formato de eventos de la simulación

Los eventos que puede contener una simulación son siempre de 2 tipos: comandos de la transacción o consultas al sistema.

2.4.1. Comandos de la transacción

Cada comando de la transacción corresponde a un *string* con toda la información para poder ejecutar dicho comando. La información se encuentra separada por punto y coma (" ; ").

- El primer elemento consiste en el identificador único de la transacción, este identificador siempre empezará con una T mayúscula y luego un número entero mayor o igual a 0. Por ejemplo, **T0**, **T4444** o **T999999999**.
- El segundo elemento consiste en el nombre del comando a ejecutar. Este siempre estará escrito en mayúscula y los posibles valores que puede tomar son: **BEGIN**, **READ**, **WRITE**, **COMMIT**, **CAN_COMMIT** o **ABORT**.
- El tercer elemento corresponde a los argumentos del comando a ejecutar. Si son 2 argumentos, estos estarán separados por una coma (",") y el orden de ellos es el mismo que el presentado en la sección de Comandos. En caso de que el comando no necesite argumentos, entonces este tercer elemento no existirá.

Para ejemplificar, a continuación se presenta un conjunto de posibles eventos.

```
1 T1;BEGIN
2 T1;READ;VARIABLE 112356
3 T1;WRITE;VAR_1,NO QUIERO MÁS
4 T1;WRITE;VAR_1,DELETE
5 T1;CAN_COMMIT;R1
6 T1;CAN_COMMIT;RRRRR-RRRRR2
7 T1;COMMIT
8 T2;BEGIN
9 T2;ABORT
```

2.4.2. Consulta

Cada consulta corresponde a un *string* con toda la información necesaria para poder ejecutar la consulta. La información se encuentra separada por punto y coma (" ; ").

- El primer elemento consiste en el identificador para las consultas que corresponde a una C mayúscula.
- El segundo elemento consiste en el nombre de la consulta a ejecutar. Este siempre estará escrito en mayúscula y los posibles valores que puede tomar son: **READ_POSSIBLE_VALUES** o **READ_COMMIT**.
- El tercer elemento corresponde al nombre de la variable consultada. Esta variable debe ser *case-sensitive*, es decir, consultar por bodoque es distinto a consultar por BODOQUE.

Para ejemplificar, a continuación se presenta un conjunto de posibles consultas:

```
1 C;READ_POSSIBLE_VALUES;guaripolo & BODOQUE
2 C;READ_COMMIT;bodoque
3 C;READ_COMMIT;BODOQUE
```

Puedes asumir que cada evento y consulta estará siempre correctamente formateada.

2.5. Resultados de la simulación

Para cada test ejecutado, se deberá crear un archivo .txt con el registro de cada consulta, cada variable existente dentro de la base de datos **final** y la cantidad de transacciones por estado posible.

Registro de Consultas

Cada vez que aparezca una instrucción de consulta, se debe registrar el valor que retorna dicha consulta. Las reglas de este registro son las siguientes:

- La primera línea del archivo siempre será el texto `##LOGS##`.
- Las siguientes n líneas corresponde al resultado de las diferentes consultas.
 - En el caso de `READ_POSSIBLE_VALUES`, será una lista en formato JSON con los valores posibles. Si esa variable no existe, la lista será vacía.
 - En el caso de `READ_COMMIT` se escribe directamente el valor que tenga la variable o la palabra `NULL`.
 - Cada línea del archivo de salida debe reflejar, en el mismo orden, los valores registrados por las consultas del archivo de entrada correspondiente.
 - En caso de $n == 0$, es decir, nunca se haya solicitado una consulta, entonces el archivo debe incluir la línea `"No hubo logs"`.

Registro de Base de datos

Una vez registrado todos los eventos de consultas, se debe registrar cada variable existente en la base de datos final, es decir, la base de datos que queda después de ejecutar cada evento del test. Las reglas de este registro son las siguientes:

- La siguiente línea debe ser el texto `##DATABASE##`
- Las siguientes k líneas corresponden a los k datos de la base de datos, donde se debe escribir el nombre de la variable, seguido de un igual (=) y finalmente el valor que tenga dicha variable finalizada toda la ejecución del test. El orden de las variables en esta sección queda a criterio del programador (orden de creación, orden alfabético, etc).
- En caso de $k == 0$, es decir, la base de datos está vacía, entonces el archivo debe incluir la línea `"No hay datos"`

Registro de Estadísticas

Finalmente, posterior a los registros de consultas y del estado final de la base de datos, la simulación debe incluir un resumen con las estadísticas de todas las transacciones existentes en el test.

Cada transacción puede terminar en uno de los siguientes estados **ABIERTA**, **ABORTADA**, **CONFIRMADA**, **EN_PREPARACION** o **INVALIDA**.

El registro de estadísticas se escribe posterior a la sección de la base de datos. Para esto, primero se debe escribir el texto `##STATS##` y luego escribir una línea por cada estado, en el siguiente formato:

```
ESTADO=[lista de identificadores de transacciones]
```

La lista debe ser una en formato JSON. Luego, el orden de los estados y de cada elemento de la lista queda a criterio del programador. Finalmente, si no existen transacciones en un estado determinado, la lista debe mostrarse vacía.

Ejemplo final

Para ejemplificar, asumiendo la siguiente situación

- La base de datos final tiene 2 variables: "saldo" con valor "1123581321" y "dueño" con valor "Hernan4444".
- Hubo 1 transacción abortada (T1), 1 inválida (T2) y dos abiertas (T3 y T4) donde una cambia el valor de 'saldo' a '4444' y otra cambia el valor de 'saldo' a '-3141529'.
- Se hizo la consulta `READ_POSSIBLE_VALUES("saldo")`, luego la consulta `READ_COMMIT("dueño")` y finalmente la consulta `READ_COMMIT("no-existe")`. Las 3 consultas se hicieron al final del test cuando solo quedaba T3 y T4 abiertas.

El archivo resultante deberá ser:

```
1 ##LOGS##
2 ["1123581321", "4444", "-3141529"]
3 Hernan4444
4 NULL
5 ##DATABASE##
6 saldo=1123581321
7 dueño=Hernan4444
8 ##STATS##
9 ABIERTA=["T3", "T4"]
10 ABORTADA=["T1"]
11 CONFIRMADA=[]
12 EN_PREPARACION=[]
13 INVALIDA=["T2"]
```

En cambio, asumiendo que la base de datos esté vacía y no se hizo ninguna consulta, y las transacciones son las mismas que el ejemplo anterior. El archivo resultante deberá ser:

```
1 ##LOGS##
2 No hubo logs
3 ##DATABASE##
4 No hay datos
5 ##STATS##
6 ABIERTA=["T3", "T4"]
7 ABORTADA=["T1"]
8 CONFIRMADA=[]
9 EN_PREPARACION=[]
10 INVALIDA=["T2"]
```

Respecto a la ruta donde se guarda el archivo y su nombre. Este se explicará con detalle en la sección [Ejecución de la tarea](#).

3. Ejemplos

En esta sección se presenta un ejemplo del archivo de entrada con diferentes consultas y su resultado final.

```
1 {
2   // Estado inicial de la base de datos
3   "DATA": {
4     "cuenta_Guaripolo": "1000", // Este sueldo es malo
5     "cuenta_Bodoque": "500"
6   },
7
8   // Servidores del banco en distintas sucursales
9   "SERVERS": ["SucursalNorte", "SucursalSur", "SucursalCentro"],
10
11   "VALIDATION": "forward",
12
13   // Secuencia de eventos
14   "TRANSACTIONS": [
15     "C;READ_POSSIBLE_VALUES;cuenta_Guaripolo",
16     "C;READ_COMMIT;cuenta_Bodoque",
17     "T1;BEGIN",
18     "T1;READ;cuenta_Guaripolo",
19     "T1;WRITE;cuenta_Guaripolo,900",
20     "T1;WRITE;cuenta_Bodoque,600",
21     "T1;WRITE;cuenta_Tulio,123",
22     "C;READ_POSSIBLE_VALUES;cuenta_Guaripolo",
23     "T1;WRITE;cuenta_Guaripolo,DELETE",
24     "C;READ_POSSIBLE_VALUES;cuenta_Guaripolo",
25     // T1 será aceptado por 2 transacciones de 3. Logra Quorum.
26     "T1;CAN_COMMIT;SucursalNorte",
27     "T1;CAN_COMMIT;SucursalCentro",
28     "T1;COMMIT",
29     "C;READ_COMMIT;cuenta_Guaripolo",
30     "C;READ_COMMIT;cuenta_Tulio"
31   ]
32 }
33
```

Resultado final

```
1 ##LOGS##
2 ["1000"]
3 500
4 ["1000", "900"]
5 ["1000"]
6 NULL
7 123
8 ##DATABASE##
9 cuenta_Bodoque=600
10 cuenta_Tulio=123
11 ##STATS##
12 ABIERTA=[]
13 EN_PREPARACION=[]
14 CONFIRMADA=["T1"]
15 ABORTADA=[]
16 INVALIDA=[]
```

4. Archivos

Para el desarrollo de esta evaluación, se provee de los siguientes archivos y carpetas:

- **Modificar** `main.py`

Este es el archivo que se ejecutará para simular la secuencia de transacciones distribuidas para un *test* específico.

El formato para ejecutar este archivo es `python3 main.py [RUTA]`, donde RUTA es el *path* relativo (desde la ubicación de `main.py`) del *test* a simular.

Un ejemplo de ejecución es `python3 main.py tests_publicos/backward/test_b_01.jsonc`.

- **No modificar** `ejecutar_tests.py`

Este es un archivo encargado de ejecutar `main.py` para todos los *tests* públicos existentes y luego contrastar la respuesta generada por `main.py` con la solución esperada.

Un ejemplo de ejecución es `python3 ejecutar_tests.py`.

Este archivo va a ser diferente al momento de corregir las tareas. Únicamente se mantendrá que este archivo utilizará `os.path.join` para garantizar que cada *path* confeccionado por *test* sea compatible al sistema operativo y que ejecutará uno por uno cada *test*.

- **No modificar** `tests_publicos/`

Esta carpeta contiene todos los archivos `.jsonc` con los diferentes *test* públicos. Esta carpeta solo existe para probar la tarea durante el desarrollo de ella, no puedes asumir que existirá al momento de corregir la tarea.

- **No modificar** `logs/`

Esta carpeta, inicialmente vacía, será utilizada para guardar todos los `.txt` con los *logs* de cada caso ejecutado. Al momento de ejecutar la tarea, el cuerpo docente se asegurará de crear esta carpeta vacía.

- **No modificar** `logs_esperados/`

Esta carpeta contiene los *logs* esperados para todos los *tests* incluidos junto a este enunciado. Lo esperado es que, luego de finalizar completamente la tarea, la carpeta `logs/` se vea exactamente igual que `logs_esperados/`.

5. Ejecución de la tarea

Cada vez que se ejecute `main.py`, este *script* recibe el *path* relativo del *test* a ejecutar. Utilizando el *path*, el *script* deberá simular el algoritmo, y además deberá correctamente el nombre del archivo `.txt` a partir de dicho *path*. El nombre del archivo de entrada debe ser utilizado para generar un archivo de salida en la carpeta `logs/` cuyo nombre siempre debe seguir el siguiente formato:

```
1 [NOMBRE_ARCHIVO_ENTRADA].txt
```

Donde `[NOMBRE_ARCHIVO_ENTRADA]` será únicamente el nombre del *test* a ejecutar sin considerar nada respecto al directorio en donde se encuentre el archivo.

Por ejemplo, asumiendo que una entrega tiene la siguiente estructura:

```
1 main.py
2 logs/
3     [vacio por ahora]
4 caso_fuera_carpeta.jsonc
5 test/
6     caso_01.jsonc
7     otra_carpeta/
8         guarcarlos_bodoque.jsonc
9     carlcetin_con_rombosman.jsonc
10     ...
```

Se ejecuta 3 veces el archivo `main.py` de la siguiente forma:

1. `python3 main.py caso_fuera_carpeta.jsonc`
2. `python3 main.py test/caso_01.jsonc`
3. `python3 main.py test/otra_carpeta/guarcarlos_bodoque.jsonc`

El directorio de la tarea deberá quedar del siguiente modo:

```
1 main.py
2 logs/
3     caso_01.txt
4     caso_fuera_carpeta.txt
5     guarcarlos_bodoque.txt
6 caso_fuera_carpeta.jsonc
7 test/
8     caso_01.jsonc
9     otra_carpeta/
10         guarcarlos_bodoque.jsonc
11     carlcetin_con_rombosman.jsonc
12     ...
```

Puedes asumir que el *path* entregado apuntará a un archivo que realmente existe haciendo directamente `open(path, encoding='UTF-8')`, es decir, no debes modificar nada de dicho *path* para poder acceder al *test* correspondiente.

Finalmente, puedes crear más archivos `.py` si lo encuentras pertinente. No obstante, debes asegurar que formas de ejecutar la tarea se mantenga tal como indica el enunciado.

6. Corrección automatizada

La corrección de toda la evaluación será **únicamente mediante test cases** para evaluar diferentes casos borde que puedan existir. En consecuencia, entregar una evaluación que no pueda ejecutarse implicará que esta será calificada con la nota mínima y sin posibilidad de corregir el error. Es responsabilidad del estudiante entregar un código ejecutable.

Adicionalmente, **no se permite utilizar ninguna librería adicional a las ya incluidas al inicio de los archivos entregados**. Si se detecta su uso o alguna tarea no se puede ejecutar porque se hizo `import` de alguna librería no autorizada, dicha tarea será evaluada con nota mínima.

Respecto a la corrección de ambos tipos de validación, se ejecutarán distintas transacciones distribuidas que pueden afectar la base de datos final. Además, durante la simulación se podrán incluir diversas Consultas para verificar que las distintas variables estén considerando correctamente el efecto de las transacciones simuladas.

Es importante recordar que los **tests públicos son solo un subconjunto de todos los casos que se evaluarán** para validar que la tarea se logra ejecutar. Queda a responsabilidad del estudiante probar la tarea con otros casos para validar la correcta implementación de la simulación.

Desglose de puntaje

- **(0.5 puntos)** Casos donde máximo hay 1 transacción es estado ABIERTA a la vez. Se cuenta con un único servidor y se probarán casos de validación *backward* y con *forward*.
- **(0.5 puntos)** Casos donde máximo hay 1 transacción es estado ABIERTA a la vez. Se cuanta con múltiples servidores y se probarán casos de validación *backward* y con *forward*.
- **(1.0 punto)** Casos con múltiples transacciones en estado ABIERTA a la vez y con un único servidor, utilizando validación *forward*.
- **(1.0 punto)** Casos con múltiples transacciones en estado ABIERTA a la vez y con un único servidor, utilizando validación *backward*.
- **(1.5 puntos)** Casos con múltiples transacciones en estado ABIERTA a la vez y con múltiples servidores, utilizando validación *forward*.
- **(1.5 puntos)** Casos con múltiples transacciones en estado ABIERTA a la vez y con múltiples servidores, utilizando validación *backward*.

Asignación de puntaje

Sea L la cantidad de `Logs` realizados en un *test*, D la cantidad de datos finales en la base de datos del mismo *test* y S la respuesta a los 5 *stats*. Se define C como $L + D + S$ que corresponde a la cantidad de consultas que se evaluarán el archivo de resultado.

Cada *test* podrá recibir uno de los siguientes valores:

- **2 puntos:** Las C consultas tienen el valor esperado.
- **1 punto:** Al menos el **70 %** de las C consultas tienen el valor esperado.
- **0 puntos:** No se alcanza el porcentaje de consultas correctas en el *test*.

Para los casos donde $L == 0$ (no hay *logs*), la única consulta a verificar es que exista el *string* correspondiente: "No hubo logs", es decir, L será 1. Del mismo modo, para los casos que $D == 0$ (base de datos vacía al final de la simulación), la única consulta a verificar es que exista el *string* correspondiente: "No hay datos", es decir, D será 1.

Es fundamental que la sección de `##LOG##` del archivo de salida contenga exactamente una línea por cada `log`, en el mismo orden en que fueron presentados en el *test*, y sin incluir líneas en blanco. La corrección se realizará comparando línea por línea con un archivo de referencia, por lo que cualquier desviación en el formato, contenido o posición —aunque la respuesta sea conceptualmente correcta— será considerada incorrecta. Asegúrense de respetar estrictamente el formato y el orden indicados para que sus respuestas puedan ser evaluadas correctamente.

Luego, para la sección de `##DATABASE##` del archivo de salida, el orden de las variables queda a decisión del programador. Para revisar esa sección, se hará la intersección de `set` entre la base de datos generada por la tarea y la base de datos esperada para determinar la cantidad de datos correctos. En caso que la base de datos del estudiante contenga más datos de la esperada, se descontará 1 consulta correcta por cada dato adicional.

Por último, para la sección de `##STATS##` del archivo de salida, el orden de las estadísticas y de cada lista queda a decisión del programador. Para revisar esa sección, se revisará que cada lista contenga exactamente los mismos elementos que la lista esperada. En caso de faltar o sobrar algún elemento, esa estadística será considerada como incorrecta.

Finalmente, cada ítem estará compuesto por un conjunto arbitrario de *tests*. Sea N la cantidad total de *tests*, se calculará el nivel de logro del ítem, el cual se utilizará para ponderar su puntaje máximo y así obtener el puntaje final correspondiente. La fórmula será la siguiente:

$$puntaje_item_x = round \left(\frac{total_puntaje_tests_logrado_x}{N \times 2} \times puntaje_maximo_item_x, 2 \right)$$

Descuentos

La evaluación está sujeta a ciertas reglas formales. El incumplimiento de alguna de ellas puede implicar descuentos automáticos o a criterio del equipo docente, según se indica a continuación:

■ PEP8 – Largo de línea

Cada línea de los archivos `.py`, sin importar la naturaleza de la línea, debe tener un **máximo de 100 caracteres**, incluyendo espacios y tabulaciones. Si al menos **una línea** supera los 100 caracteres (es decir, tiene 101 o más), se descontarán 0.2 puntos de la nota final, incluso si la línea excedente contiene solo espacios, tabulaciones o comentarios.

■ Modularización

Cada archivo `.py` puede contener un **máximo de 400 líneas**, incluyendo código, comentarios y líneas en blanco. Si al menos **un archivo supera este límite** (401 líneas o más), se descontarán 0.2 puntos de la nota final, sin importar el contenido de la línea extra.

■ Formato de Entrega

Si el estudiante entrega una solución funcional, pero esta no respeta alguna regla del enunciado o del curso, el descuento será determinado por el equipo docente, y podrá variar entre 0.1 puntos y hasta 6.0 puntos, dependiendo de la gravedad del error.

Por ejemplo, si un estudiante utiliza una librería externa que no estaba permitida y debido a ello, el código no se ejecuta correctamente en el computador del equipo docente, la penalización será de 6.0 puntos.

En cambio, si el estudiante incluye un `import` de una librería no permitida que impide la ejecución del código en los computadores del equipo docente, pero el código no hace uso real de dicha librería y solamente eliminando el `import` permite que el código funcione. El equipo docente puede evaluar la posibilidad de eliminar dicho `import` para ejecutar la tarea, pero se aplicaría un descuento según lo definido anteriormente.

7. Entregables

Se espera que el entregable sea un archivo ZIP que contenga como mínimo el archivo **main.py**. En caso de crear archivos adicionales para modularizar la tarea, estos también deben estar incluidos en el ZIP.

El archivo `.zip` entregado también debe incluir un archivo `README.md` que contenga:

- Todas las **citas** de material externo utilizado.
- Una explicación clara y completa del uso de cualquier **herramienta generadora de código** (por ejemplo: asistentes de inteligencia artificial, generadores automáticos, etc.).

Si el archivo `README.md` está ausente o no incluye alguna de las referencias necesarias, se asumirá que **todo el contenido entregado sin referencias es de autoría exclusiva del estudiante**.

Advertencia: Si se detecta uso de material externo o herramientas generativas sin que estén debidamente citadas en el `README.md`, se considerará una **falta a la integridad académica**, y se aplicarán las sanciones correspondientes según las políticas establecidas en el programa del curso.

En caso de trabajar en parejas, solo un integrante debe entregar el trabajo. En caso que ambos entreguen, se revisará cualquiera de los 2 entregas sin poder apelar a dicha decisión.

Finalmente, no se aceptarán entregas en formatos distintos al indicado (por ejemplo, archivos `.ipynb`, `.pyc`, etc.). Si no se entrega el archivo ZIP o se entrega en un formato diferente, la entrega no será revisada y se asignará la nota mínima.

8. Dudas

Cualquier duda que tengas sobre esta evaluación, prefiere publicarla en el [Syllabus](#) correspondiente a esta evaluación. También, siente la libertad de responder dudas de tus pares si crees que conoces la respuesta. En caso de tener dudas que impliquen mostrar tu solución o partes de ella, no utilices este medio de consulta. Para estos casos, envía un correo al cuerpo docente o muestra tu solución solo en reunión personal (remota o presencial) cuando te reúnas con algún miembro del cuerpo docente.

9. Política de atraso

Existe la posibilidad de entregar esta evaluación con hasta **2 días de atraso** a partir de la fecha de entrega definida en el enunciado. En la eventualidad de entregar pasada la fecha de entrega, se aplicará una **reducción** a la nota máxima que podrás obtener en esta evaluación.

Si el trabajo se realiza en parejas, se corregirá aquella entregada con atraso y la reducción de nota aplicará a ambos miembros.

De haber atraso, **la nota máxima a obtener** se reduce en **1 punto (10 décimas)** por cada día de atraso. Cada día de atraso se determina como el techo de días de atraso. Por lo tanto, en caso de entregas atrasadas, la nota final de la tarea se calcula mediante la siguiente fórmula:

$$\text{mín}(7 - (0.5 \times \text{días_atraso}), \text{nota_obtenida}) - \text{descuento_total})$$

Por otro lado, entregas con más de 2 días (48 hrs) de atraso no serán recibidas y serán evaluadas con la **calificación mínima (1.0)**.

10. Integridad académica y código de honor

Recuerde que esta evaluación está alineada al Código de Honor de la Universidad y se espera un comportamiento íntegro del estudiantado al momento de desarrollar. Dado lo anterior, se **enfatisa** que la solución de esta evaluación debe ser producto de un máximo 2 estudiantes que trabajaron en conjunto. y que se inscribieron oportunamente en el formulario.

Finalmente, respecto al uso de herramientas generadoras de código. Estas están permitidas siempre y cuando se referencie correctamente cuándo es utilizado. Junto a lo anterior, se espera que **toda respuesta entregada por una herramienta generadora de código o de IA pase por un proceso de análisis crítico y modificación antes de ser incluida en una evaluación**. En caso de entregar una tarea donde se identifique que solo hubo un proceso de copiar la respuesta dada por una herramienta generadora de código o de IA, esto no será aceptado y será considerado como una falta a la integridad académica.